Response to Editor's and Reviewers' Comments
CYSE-D-26-01344

Evidence-Guided Crash Adjudication for LLM-assisted Fuzzing of Third-party Open-source Libraries in Connected Vehicles

Qingguan Gao, Ying Zhang, Letian Sha, and Huiran Yang

November 8, 2026

November 8, 2026

Dear Editor,

Please find enclosed the revision of our paper "Evidence-Guided Crash Adjudication for LLM-assisted Fuzzing of Third-party Open-source Libraries in Connected Vehicles" with manuscript ID CYSE-D-26-01344.

We would like to thank you for handling the review process of our paper. We are also indebted to the reviewers for their helpful comments. A detailed point-by-point response to the comments is given below.

Yours sincerely,

Qingguan Gao, Ying Zhang, Letian Sha, and Huiran Yang

Note: To help the legibility of the remainder of this response letter, all the editor's and reviewers' comments and questions are typeset in *italic font*. Our responses are written in plain font. Rephrased sentences and added content in the revised manuscript are typeset in blue color. Quoted manuscript excerpts are also shown in blue. Section, table, and figure references below refer to the revised manuscript.

Cybersecurity
Manuscript ID: CYSE-D-26-01344
Authors' Response to the Editor

Comment 1: *Please include a point-by-point response within the "Response to Reviewers" box in the submission system and highlight (with "tracked changes"/coloured/underlines/ highlighted text) all changes made when revising the manuscript. Please ensure you describe additional experiments that were carried out and include a detailed rebuttal of any criticisms or requested revisions that you disagree with. Please also ensure that your revised manuscript conforms to the journal style, which can be found in the Submission Guidelines on the journal homepage.*

Response: Thank you for the opportunity to revise our manuscript. We provide a point-by-point response to all nine comments from Reviewer 1 and all six comments from Reviewer 2 below. The revised manuscript incorporates the corresponding text, tables, figures, and additional experimental results. Revised text and table content are highlighted in blue.

Cybersecurity
Manuscript ID: CYSE-D-26-01344
Authors' Response to Reviewer 1

Comment 1: *The paper correctly identifies the core issue that LLM-generated drivers are untrusted, but does not clearly delineate the specific scope of "untrusted". For example, drivers may be maliciously crafted, unintentionally erroneous, or misused due to insufficient API understanding—should attribution strategies differ across these three cases? The authors are advised to distinguish "malicious drivers" from "incompetent drivers" in the threat model and discuss the applicability of rule-guided crash attribution to these two categories.*

Response: Thank you for pointing out the ambiguity. This study focuses on non-malicious drivers that may be faulty, i.e., LLM-generated fuzz drivers that contain unintended implementation errors or misuse target APIs. The assumption is not that every generated driver is faulty, but that a crash report cannot be taken as evidence of a target defect until the driver has been ruled out. Our goal is to discover defects in target libraries through fuzz testing using these generated drivers. However, the drivers themselves may be faulty, making it necessary to distinguish driver-induced crashes from actual target-library defects. Deliberately malicious drivers are therefore outside the scope of this study. We have clarified this scope in Section 3.1 and stated that rule-guided attribution is designed for unintended driver errors and API misuse.

The above revisions have been incorporated into Section 1 (Introduction), Section 2.1 (LLM-Assisted Fuzzing), and Section 3.1 (LLM-Assisted Fuzzing Framework). The corresponding excerpts are reproduced below.

> Section 3.1, LLM-Assisted Fuzzing Framework (final paragraph)
> "In the workflow described above, generated drivers are assumed to be non-malicious but potentially faulty: they may contain unintentional implementation errors or may misuse the target APIs, and we refer to such a driver as an incompetent driver. The assumption is not that every generated driver is faulty, but that a crash report cannot be taken as evidence of a target defect until the driver has been ruled out. A faulty driver may omit required initialization, construct invalid arguments, or otherwise violate API contracts because of unintentional generation errors or insufficient API understanding. We assume that the sanitizer reports, crash artifacts, and source code used for

analysis have not been deliberately tampered with. Under these assumptions, rule-guided crash attribution uses sanitizer signals and stack-frame roles to screen likely driver-induced failures. Deliberately malicious drivers, including those that forge evidence or attempt to evade adjudication, are outside the scope of this work."

Section 1, Introduction
"To address this problem, we propose CrashSage, an evidence-guided crash-adjudication pipeline tailored for LLM-assisted fuzzing with incompetent generated drivers."

Section 2.1, LLM-Assisted Fuzzing
"Once a generated driver compiles and produces crashes, crash reports are usually treated as fuzzing outcomes rather than as evidence that must be adjudicated under the assumption that the harness may be incompetent."

Comment 2: *Table 8 defines three ground-truth categories—"confirmed target," "confirmed non-target," and "potential target"—but does not specify the number of annotators, inter-annotator agreement (e.g., Cohen's Kappa), or whether annotators were aware of CrashSage's outputs. Given that "confirmed target" requires correspondence with public evidence such as CVEs/advisories/patches, and "potential target" is excluded from binary metrics, the authors are advised to supplement labeling protocol details and discuss whether excluding 145 potential targets may lead to optimistic estimation of method performance.*

Response: Thank you for this helpful comment on the labeling protocol and the treatment of the 145 potential-target reports. We have replaced the label definitions in the revised manuscript and supplemented the annotation procedure and its statistics.

This study uses two labels, target and non-target. A target label means that the evidence is sufficient to determine that the crash corresponds to a vulnerability in the target library, and a non-target label means that the evidence is sufficient to determine that it does not. The previous three-category scheme, namely "confirmed target", "confirmed non-target", and "potential target", is no longer used, and the earlier requirement that a target label be backed by public evidence has been removed: public vulnerability material and manual verification are two routes to the same evidence rather than a condition on the label. A report without a matching CVE is therefore neither assigned non-target by default nor excluded from the evaluation; it is resolved by manual verification.

The 876 reports of the original submission and the 196 reports added in this revision were labeled by matching each crash to public vulnerability material. For each report, the correspondence was established by checking the target version, the crash signature, the stack trace path, and the triggering input, and a non-target label was determined from the verification of the relevant code and of the triggering process rather than from the mere absence of a matching CVE.

For the 145 reports that lacked sufficient public vulnerability evidence, we conducted an additional manual audit. Three people with C/C++ memory-safety analysis experience took part. These 145 reports form 32 representative groups, and the annotation unit was the group: the two reviewers reproduced the representative crash from the triggering input, checked the crash signature and the stack trace path, and, using the API contracts and the relevant source code, judged whether the crash exposes a vulnerability in the target library, without seeing CrashSage outputs. Reviewer A labeled 31 groups and Reviewer B labeled all 32, so 31 groups carry paired labels; the reviewers agreed on 22 of them, which gives a raw agreement of 70.97% (22 of 31 groups) and a Cohen's kappa of 0.413. The unit of these two statistics is the representative group and the denominator is the 31 groups with paired labels, not the 145 reports, because a group was judged as one analysis unit. The third person re-examined the nine disagreement groups and the one unpaired group, reviewed the basis of both annotations, and performed the necessary code inspection and reproduction to determine the final label. Each group label was then applied to the member reports, and all 26 reports that finally received a target label were additionally verified one by one against their own crash evidence, so the target labels do not rest on the grouping alone. The 119 non-target reports keep the group label, because the members of a group share one crash site and one call path and were therefore judged on the same evidence. In total, 26 reports were labeled target and 119 were labeled non-target.

The 876 reports of the original submission, the 196 reports added in this revision, and the 145 audited reports therefore account for all 1,217 reports. The revised evaluation includes all of them, comprising 215 target and 1,002 non-target reports, and no report is excluded from the binary metrics. All experimental configurations use this same fixed ground truth, so the previously pending reports are counted rather than set aside. We have added the label definitions, the annotation procedure, and the sample statistics to Section 5.1.

The above revisions have been incorporated into Section 5.1 (Experimental Setup) and Table 8. The corresponding excerpts are reproduced below.

Section 5.1, Experimental Setup

"Each crash report receives a binary evaluation label: target or non-target. A target label means that the evidence is sufficient to determine that the crash corresponds to a vulnerability in the target library, and a non-target label means that the evidence is sufficient to determine that it does not. The evidence comes from public vulnerability material when a CVE record, an official advisory, or a public issue or patch record can be matched to the crash, and otherwise from manual verification of the crash against the target source and a local reproduction. The absence of a matching CVE is not by itself a reason for a non-target label, and a report that still lacks evidence is not assigned a label until the evidence is obtained; every report in the final evaluation therefore has a label supported by evidence. The final evaluation contains 215 target reports and 1,002 non-target reports across all 1,217 reports, as summarized in Table 8. Precision, recall, and F1 are computed over this complete report population."

Section 5.1, Experimental Setup

"The 876 reports of the original submission and the 196 reports added in this revision were labeled by matching each crash to public vulnerability material. For each report, the correspondence was established by checking the target version, the crash signature, the stack trace path, and the triggering input, and a non-target label was determined from the verification of the relevant code and of the triggering process rather than from the mere absence of a matching CVE.

For the 145 reports that lacked sufficient public vulnerability evidence, we conducted an additional manual audit. Three people with C/C++ memory-safety analysis experience took part. These 145 reports form 32 representative groups, and the annotation unit was the group: the two reviewers reproduced the representative crash from the triggering input, checked the crash signature and the stack trace path, and, using the API contracts and the relevant source code, judged whether the crash exposes a vulnerability in the target library, without seeing CrashSage outputs. Reviewer A labeled 31 groups and Reviewer B labeled all 32, so 31 groups carry paired labels; the reviewers agreed on 22 of them, which gives a raw agreement of 70.97% (22 of 31 groups) and a Cohen's kappa of 0.413. The unit of these two statistics is the representative group and the denominator is the 31 groups with paired labels, not the 145 reports, because a group was judged as one analysis unit. The third person re-examined the nine disagreement groups and the one unpaired group, reviewed the basis of both annotations, and performed the necessary code inspection and

reproduction to determine the final label. Each group label was then applied to the member reports, and all 26 reports that finally received a target label were additionally verified one by one against their own crash evidence, so the target labels do not rest on the grouping alone. The 119 non-target reports keep the group label, because the members of a group share one crash site and one call path and were therefore judged on the same evidence. In total, 26 reports were labeled target and 119 were labeled non-target.

The 876 reports of the original submission, the 196 reports added in this revision, and the 145 audited reports therefore account for all 1,217 reports. No report is excluded from the binary metrics, and all experimental configurations use this same fixed ground truth."

Table I: Ground-truth accounting for the final binary evaluation (revised Table 8).

Binary label	Reports
Target	215
Non-target	1,002
Total evaluated reports	1,217

Comment 3: *Section 4.3 performs representative selection via exact and stack signatures, and Section 5.3 states that "each representative diagnosis and ground-truth label are propagated to its duplicate reports before computing TP, FP, and FN." However, the transitivity of signature matching (i.e., whether A matches B and B matches C implies A matches C) and the risk of error propagation (a misjudged representative causing the entire group to be misjudged) are not discussed. The authors are advised to supplement: (1) the equivalence class construction algorithm for signature matching; (2) case analysis or quantitative evaluation of group-level misjudgment caused by representative misjudgment.*

Response: Thank you for this careful reading. We have clarified the representative-selection algorithm, corrected the description of label propagation, and added an analysis of the effect of representative misjudgment.

First, only the diagnosis of a representative is propagated to its duplicate reports; the ground-truth labels are not propagated. Every report keeps the ground-truth label determined by the annotation protocol, and the propagated prediction is compared with that report's own label to compute TP, FP, TN, and FN. Errors introduced by propagation therefore enter the final metrics. The statement on ground-truth propagation in the previous version has been corrected.

Second, Algorithm 1 uses a representative-based matching strategy rather than constructing strict equivalence classes through a transitive closure. The algorithm compares each report, in discovery order, with the existing representatives by exact or stack signature; when the matching condition is satisfied, the report is associated with the first matching representative, and otherwise it becomes a new representative. Reports are therefore not merged merely because of an indirect match, and the procedure does not require the matching relation to be transitive. Representativeness follows from the signature-matching condition, and "first match" is used only to determine a unique mapping.

Third, to quantify the effect of representative misjudgment, we flip the prediction of each of the 96 representatives in turn and recompute the report-level metrics from the saved mapping. The maximum absolute change in overall F1 is 0.0174, in the most influential case F1 changes from 0.8379 to 0.8205, and no flip changes more than 7 of the 1,217 report-level decisions. A change in one representative affects only its own group, while the other groups remain unchanged, so the overall effect on this dataset is limited. Representative selection also reduces LLM submissions by 82.02%, and in this evaluation V1 and V2 achieve identical report-level attribution metrics. We have added these clarifications to the revised manuscript.

The above revisions have been incorporated into Section 4.3 (Representative Selection) and Section 5.4 (Evaluation of Representative Selection). The corresponding excerpts are reproduced below.

> Section 4.3, Representative Selection
>
> "CrashSage processes the retained crashes in discovery order and compares each crash with the representatives selected so far. A matched crash is linked to the corresponding representative, whereas an unmatched crash becomes a new representative. Algorithm 1 summarizes this procedure.
>
> The procedure therefore does not construct equivalence classes through a transitive closure. A crash is associated with the first matching representative, and two crashes that match the same representative are not thereby declared equivalent. Each crash is mapped to at most one representative, and the mapping is stored explicitly rather than inferred from the match relation."

Section 5.4, Evaluation of Representative Selection

"Figure 4 shows that V1 submits all 534 reports retained after rule-guided crash attribution to the LLM, whereas V2 submits only 96 representatives and removes 438 repeated or redundant reports. Representative selection therefore reduces LLM submissions by 82.02% relative to V1. Only the prediction of each representative is propagated through the saved mapping, while every report retains its own ground-truth label and TP, FP, TN, and FN are computed against that label. In this evaluation, V1 and V2 achieve identical report-level attribution metrics, while V2 requires substantially fewer LLM submissions. Although each V2 request contains richer analysis content than an individual raw report, reducing the number of LLM submissions from 534 in V1 to 96 in V2 substantially lowers the overall LLM invocation cost as measured by the number of LLM submissions."

Section 5.4, Evaluation of Representative Selection

"To evaluate group-level error propagation, we flip each of the 96 representative predictions individually and recompute the report-level metrics from the saved mapping and the archived V4 predictions. Over the 96 flips, the maximum absolute F1 change is 0.0174; the worst case reduces F1 from 0.8379 to 0.8205, and no flip changes more than 7 of the 1,217 report-level decisions."

Comment 4: *Table 12 reports the runtime of rule-guided crash attribution and representative selection (<10 ms/report and <5 ms/report, respectively), but does not report the runtime of crash-context reconstruction and LLM analysis. Section 5.3 mentions that "runtime context reconstruction and LLM analysis are the main sources of latency," but specific data are lacking. The authors are advised to supplement: (1) detailed time breakdown of crash-context reconstruction (including seed recovery, replay, and GDB collection); (2) total end-to-end latency on the 57 representative samples; (3) discussion of scalability to larger-scale libraries (e.g., those with more APIs).*

Response: Thank you for this helpful suggestion. We have revised Table 13 and the accompanying runtime discussion to address the three points. Because the implementation and evaluation were rechecked and expanded during revision, the historical values in this comment are superseded by the recomputed results below.

First, regarding the detailed breakdown of crash-context reconstruction, we now separately report crash record lookup, seed recovery and binary resolution, static context extraction, direct replay, and Docker/GDB context collection. Their mean times are 0.32

s, 0.18 ms, 1.18 ms, 2.44 s, and 18.96 s per representative, respectively, and these five components sum to the reported mean context reconstruction time of 21.72 s. Crash record lookup locates the archived crash record and loads its metadata, seed recovery locates the crashing input, and binary resolution identifies the corresponding executable. Docker/GDB collection remains the dominant substep. We also report evidence packaging and prompt construction at 0.34 s and LLM analysis at 6.63 s per representative. The revised table presents the mean time for each stage, and the accompanying text explains the replay/debugger attempt limits and the inclusion of failed and timed-out attempts.

The above revisions have been incorporated into Section 5.4 (Evaluation of Representative Selection) and Table 13. The corresponding excerpts are reproduced below.

Table II: Mean runtime of the analysis stages per representative (revised Table 13).

Stage	Mean time
Sanitizer parsing	0.67 ms
Rule-guided attribution	0.10 ms
Representative selection	0.01 ms
Crash record lookup	0.32 s
Seed recovery and binary resolution	0.18 ms
Static context extraction	1.18 ms
Direct replay	2.44 s
Docker/GDB context collection	18.96 s
Context reconstruction (subtotal of the five rows above)	21.72 s
Evidence packaging and prompt construction	0.34 s
LLM analysis	6.63 s
End-to-end representative analysis	28.69 s

Second, regarding end-to-end latency on the representative samples, the expanded evaluation dataset contains 96 representatives, compared with 57 in the original manuscript. We have therefore updated the timing results to cover all 96 representatives. Their mean end-to-end latency is 28.69 s per representative, including context reconstruction, evidence packaging and prompt construction, and LLM analysis. Because the context measurements were collected serially, the 96 representatives require approximately 45.9 minutes of processing in total, which is the sum of the per-representative mean over the evaluated representatives. This figure is an estimate of accumulated analysis time, not a measured wall-clock time of one end-to-end run, because the context measurements and the archived LLM responses were collected separately and are combined through the per-representative mean. This result is reported in both the revised table and the accompanying text.

Third, regarding scalability to larger libraries beyond the evaluated targets, we have expanded the discussion using cross-project measurements. The evaluated target API sets

range from 12 to 60. For example, cJSON and libpcap cover 60 and 30 target APIs and each requires only four LLM submissions, so broader tested API coverage can still produce a compact representative workload. CrashSage constructs evidence around selected crash sites and relevant execution paths, so increasing the API set does not require submitting the entire library to the model. When additional API interactions produce new representative groups, further reconstruction and model calls are required, and the overall cost depends mainly on the number of representatives and on the replay, debugger, and LLM cost of each representative. Representative selection reduces the additional analysis caused by repeated reports.

> Section 5.4, Evaluation of Representative Selection
> "The mean end-to-end latency across the 96 representatives is 28.69 s, covering context reconstruction, evidence packaging and prompt construction, and LLM analysis. Context and prompt times come from the timing measurements, while LLM latency comes from the archived responses for the same representatives. Because the context measurements were collected serially, the 96 representatives require approximately 45.9 minutes of processing in total, which is the sum of the per-representative mean over the evaluated representatives. This figure is an estimate of accumulated analysis time rather than a measured wall-clock time of one end-to-end run, because the context measurements and the archived LLM responses were collected separately and are combined through the per-representative mean."

Comment 5: *Table 7 defines V0 as the "CKGFuzzer baseline," but CKGFuzzer is itself a fuzz driver generation system whose crash adjudication capability is not its design goal. Is it fair to use it as a crash adjudication baseline? The authors are advised to supplement: (1) comparative experiments with dedicated crash deduplication/localization methods (e.g., Igor, ReBucket, CrashLocator); (2) comparison with a baseline that directly uses LLM to classify raw reports; (3) discussion of why CKGFuzzer was chosen over other LLM-assisted fuzzing systems (e.g., PromptFuzz, Fuzz4All) as the baseline.*

Response: Thank you for this valuable comment. The comment raises three points: whether CKGFuzzer is a fair crash-adjudication baseline, which dedicated methods and which raw-report baseline the comparison should include, and why CKGFuzzer was chosen over other LLM-assisted fuzzing systems. We address them in turn.

On the fairness of the baseline, CKGFuzzer enters the comparison as the complete crash-cause analysis pipeline rather than as a driver generator alone. Its crash-analysis

stage is active in V0, so the comparison does not credit CrashSage with adjudication that the baseline already performs. All variants share the same upstream driver generation and fuzzing workflow and the same crash reports, and the revised manuscript identifies which CKGFuzzer components are retained and which components are added. The V0–V4 comparison therefore measures the four added stages on top of that pipeline.

On dedicated methods, we select ReBucket as the external reference for the selection stage, because it clusters crashes by call-stack similarity, which is directly related to reducing repeated analysis, and it can be applied to the same report set. On the 534 reports retained after rule-guided crash attribution, ReBucket selects 103 cluster representatives and CrashSage selects 96, corresponding to reductions of 80.71% and 82.02%, and the pairwise Jaccard agreement is 0.9824.

On the raw-report LLM baseline, we pass each of the 1,217 reports directly to the same model with a minimal direct instruction, without filtering, representative selection, reconstructed context, or the differential prompt. It performs the same binary decision as CrashSage and obtains Precision 0.2010, Recall 0.5488, and F1 0.2943, against 0.7654, 0.9256, and 0.8379 for CrashSage.

On the choice of CKGFuzzer, it provides the driver generation, fuzzing, and crash-analysis workflow on which this study builds. PromptFuzz and Fuzz4All target input or driver generation and do not expose the crash-analysis interface used here. Replacing CKGFuzzer with either system would change the crash corpus, the build and sanitizer settings, and the labeling conditions, so the comparison would conflate upstream fuzzing effectiveness with downstream adjudication quality.

The above revisions have been incorporated into Section 5.1 (Experimental Setup), Section 5.2 (Comparison with Raw-Only LLM Analysis), and Table 7. The corresponding excerpts are reproduced below.

> Section 5.1, Experimental Setup
> "All variants share the same base configuration, which is the crash-analysis stage of CKGFuzzer: the crash report, the generated driver and the APIs it calls, the error patterns extracted from their source, and the retrieved CWE entries. V0 therefore runs the crash-analysis stage of CKGFuzzer with its original inputs unchanged and adds none of the four CrashSage stages. V1–V4 keep this configuration unchanged and add the stages listed in Table 7. The V0–V4 comparison therefore measures the incremental effect of these added stages while holding the fuzz-driver generation and fuzzing workflow of CKGFuzzer fixed and evaluating all variants on the same crash reports."

Table III: Evaluation configurations (revised Table 7).

Variant	Base configuration	Rule-guided filtering	Representative selection	Reconstruction context	Final prompt
V0 (CKGFuzzer baseline)	Yes	No	No	No	Clean direct prompt
V1	Yes	Yes	No	No	Clean direct prompt
V2	Yes	Yes	Yes	No	Clean direct prompt
V3	Yes	Yes	Yes	Yes	Clean direct prompt
V4 (CrashSage)	Yes	Yes	Yes	Yes	Differential prompt

Section 5.2, Comparison with Raw-Only LLM Analysis

"To evaluate raw-report analysis, we compare a raw-only LLM baseline with the complete CrashSage V4 on the 1,217 crash reports included in the main evaluation. The raw-only baseline passes each original sanitizer report to `qwen3-coder-flash` at temperature 0 with a minimal attribution instruction, without filtering, representative selection, reconstructed context, or the differential prompt."

Section 5.1, Experimental Setup

"This workflow is adopted because CKGFuzzer couples driver generation with a crash-analysis stage, so the adjudication pipeline can be extended without replacing the upstream fuzzing process; systems that target only input or driver generation do not expose a comparable crash-analysis interface."

Comment 6: *Section 5.1 uses Qwen3-Coder-Plus as the primary model, and Section 5.5 compares Qwen3.7-Max and Qwen-Flash. However, it is not explained why the Qwen series was chosen over other commercial models such as GPT-4 or Claude. Additionally, cloud API version updates may render results irreproducible. The authors are advised to supplement: (1) rationale for model selection (cost, availability, code understanding capability, etc.); (2) model version numbers and access dates; (3) whether open-source local models (e.g., CodeLlama, DeepSeek-Coder) were considered to enhance reproducibility.*

Response: Thank you for this valuable comment. We answer the three questions on model choice, version identification, and locally deployed models. The cross-model evaluation, the repeated runs, and the prompt-component ablation are reported in our response to Comment 7.

On the first question, we evaluate V4 with six backends while holding the input evidence, the representative mapping, and the ground-truth labels fixed. The six backends

cover the Qwen, DeepSeek, GPT, and GLM families, and F1 ranges from 0.8240 to 0.8379 across them, so the reported behavior is not specific to the Qwen series and the choice of the Qwen family does not drive the results. The default backend of CrashSage is `qwen3-coder-flash`, selected on four practical criteria: general and code-analysis capability on C/C++ crash evidence, a context window large enough to hold the evidence package, service stability and response latency, and API cost. We now state these criteria alongside the evaluated model configuration.

On the second question, we report the API model identifiers, the endpoint, the access dates, and the decoding parameters, and we retain the raw model responses in the experiment archive. The previous version reported Qwen3-Coder-Plus. In this revision every configuration, including the repeated runs and the prompt ablation, was executed with `qwen3-coder-flash` on the expanded 1,217-report set, so all reported numbers come from one backend and one access window, and the earlier Qwen3-Coder-Plus results are superseded rather than combined with the new ones. Keeping one backend across all configurations avoids a comparison in which different variants are scored by different models. All configurations use `temperature=0`. The workspace identifier in the endpoint URL is redacted in the manuscript for anonymous review and is recorded in full in the experiment archive. The backend names in Table 18 are the model identifiers exposed by each provider's API, which is the identifier we queried; where a provider serves a model under an alias, the alias is what we report.

On the third question, thank you for this suggestion. We considered locally deployed open-source models. Given the hardware and context-budget constraints of this study, we used hosted backends to keep the evidence budget consistent across evaluated models. We appreciate the stronger control over model revisions offered by a fixed local deployment, and we now state this as a limitation in Section 6.

The above revisions have been incorporated into Section 5.6 (Evaluation of Final-Prompt Sensitivity and LLM), Section 6 (Limitations), and Table 18. The corresponding excerpts are reproduced below.

> Section 5.6, Evaluation of Final-Prompt Sensitivity and LLM
>
> "The primary CrashSage-V4 model is `qwen3-coder-flash`, accessed on 1 November 2026 through the OpenAI-compatible endpoint `https://[workspace-id].cn-beijing.maas.aliyuncs.com/compatible-mode/v1`; the workspace identifier in this URL is redacted for anonymous review and is recorded in full in the experiment archive. This model was selected on four engineering metrics: general and code-analysis capability, long-context capacity, service stability and response latency, and API cost."

Section 5.6, Evaluation of Final-Prompt Sensitivity and LLM

"The alternative backends are `qwen-flash`, `qwen3.8-flash`, `deepseek-flash`, `gpt-5.6-sol`, and `glm-5.3-flash`, accessed between 30 October and 1 November 2026. Each backend was accessed through the API provider under its exact model identifier. Model identifiers and access dates are reported so that each evaluated configuration can be identified. All configurations use `temperature=0`."

Table IV: Validation results of different LLM backends under the same V4 configuration (revised Table 18).

LLM Backend	Precision	Recall	F1
Qwen3-Coder-Flash (default V4)	0.7654	0.9256	0.8379
Qwen-Flash	0.7425	0.9256	0.8240
Qwen3.8-Flash	0.7636	0.9163	0.8330
DeepSeek-Flash	0.7645	0.9209	0.8354
GPT-5.6-Sol	0.7645	0.9209	0.8354
GLM-5.3-Flash	0.7606	0.9163	0.8312

Section 6, Limitations

"All experiments use hosted LLM backends. A fixed local deployment would give stronger control over model revisions, but the hardware and context budget of this study did not allow one, so reproducibility relies on the reported model identifiers, access dates, and archived raw responses."

Section 5.6, Evaluation of Final-Prompt Sensitivity and LLM

"Run-to-run stability. The four prompt configurations defined in the nested prompt ablation were frozen before evaluation and shared across projects. No project names, expected CVE identifiers, or ground-truth labels were inserted into the templates. We repeated each configuration six times at temperature 0 and report decision variability at both the run and report levels in Table 16. The Precision, Recall, and F1 columns of that table report one run per configuration: the run with the median F1 of the six repetitions for P1 and P2, and the formal V3 and V4 endpoints for P0 and P3. The six-run mean and standard deviation are computed over the same six repetitions for all four configurations, and these means are the values we compare across configurations. The intervals describe run-to-run variation within a configuration; they are not a test between configurations, so we do not use their overlap to judge significance."

Comment 7: *Table 15 shows that differential prompting improves recall from 0.829 to 0.952 but slightly decreases precision. Table 16 shows that the prompt significantly improves cJSON and json-c but not tinyxml2. This indicates that the effect of prompt design is project-dependent. The authors are advised to supplement: (1) stability experiments of differential prompting across different LLM models; (2) whether there is a risk of prompt overfitting (i.e., tuning prompts for specific libraries); (3) ablation experiments on prompt templates to identify key prompt components.*

Response: Thank you for this valuable comment. The differential prompt asks the model to compare two explanations, namely driver misuse and a target-library defect, because an in-library crash can be caused either by the driver violating an API contract or by a vulnerability in the target library. We have added experiments that analyse the stability, the project-level differences, and the component roles of this design, and we address the three points below.

First, stability across models. We evaluate the complete V4 pipeline with six backends, including three non-Qwen backends, `deepseek-flash`, `gpt-5.6-sol`, and `glm-5.3-flash`, while keeping the representatives, reconstructed evidence, prompt, mapping, and ground truth fixed. F1 ranges from 0.8240 to 0.8379 and recall from 0.9163 to 0.9256. Because this experiment runs the complete V4 configuration only, it shows that V4 performance is close across backends; it does not separately verify the V3-to-V4 prompt gain on each backend. We also repeat each prompt configuration six times at temperature 0. Mean F1 is 0.7788, 0.8095, 0.8260, and 0.8295 for P0–P3, respectively; standard deviations range from 0.0051 to 0.0100, and decisions vary for 9–23 of the 1,217 reports across runs.

Second, safeguards against overfitting and project-dependent effects. The prompt template was frozen before evaluation and shared across all libraries. It contains no library-specific instructions, CVE identifiers, or ground-truth labels; project evidence is supplied separately. Only cJSON, expat, and jansson change between V3 and V4; the other eleven projects retain the same results.

The large cJSON improvement follows from its representative mapping. Of 78 reports, six are target and 72 non-target. Filtering retains 14 reports, which map to four representatives; all six targets belong to one group comprising the representative and five duplicates. V3 classifies this representative as non-target, V4 as target, using the same evidence and mapping. Propagation therefore changes TP from 0 to 6 and FN from 6 to 0, while FP remains 0 and TN remains 72, raising F1 from 0.0000 to 1.0000. In the selected P1 run, this representative already changes after the two-hypothesis comparison is added. The large gain thus reflects one improved decision covering all six positives, rather than six independent model decisions. Other projects have different group distributions and

starting points, so their gains differ as well.

Third, component ablation. We compare four nested variants using the same representatives and evidence. P0 is the V3 Clean direct prompt; P1 adds the two-hypothesis comparison; P2 adds precondition checking and `evidence_rationale`; P3 adds boundary rules and corresponds to V4. The comparison is based on the mean F1 of six runs, which is computed for all four configurations and is 0.7788, 0.8095, 0.8260, and 0.8295, respectively. The single-run columns of Table 16 report one run per configuration, namely the median-F1 run for P1 and P2 and the formal V3 and V4 endpoints for P0 and P3; we use them as a case analysis and state their provenance in the caption.

The decision counts of that run locate each step in the error space. P1 increases TP from 186 to 199 and reduces FN from 29 to 16, while FP remains 75. Recall therefore rises from 0.8651 to 0.9256 without increasing the false-positive count. P2 and P3 then reduce FP to 68 and 61, while TP stays at 199. The later checks remain useful after recall plateaus: they refine positive attributions through caller-precondition and supporting-evidence checks. In this nested ablation, the two-hypothesis comparison accounts for the main recall gain, while precondition auditing and boundary rules improve precision. The gap between P0 and P1 (0.0307 in mean F1) is larger than the six-run standard deviation of either configuration, whereas the gap between P2 and P3 (0.0035) is smaller than that spread. We performed no paired test between configurations, so we report the ordering of the means and do not claim that each individual step is statistically significant.

The above revisions have been incorporated into Section 5.6 (Evaluation of Final-Prompt Sensitivity and LLM), Table 16, and Table 17. The corresponding excerpts are reproduced below.

Section 5.6, Evaluation of Final-Prompt Sensitivity and LLM

"Table 16 shows that mean F1 over the six runs rises at every step, from 0.7788 to 0.8095, 0.8260, and 0.8295, and that the same ordering holds in the runs reported in the table. The three steps do not act on the same error type. The direct prompt yields more conservative positive decisions because it requests a final attribution without requiring the model to compare target-component-defect and driver-misuse hypotheses. In this nested ablation, the two-hypothesis comparison accounts for the main recall gain, and recall stops rising after it. The two later components remove false positives only, so precision keeps rising while recall stays fixed, which places their contribution in precision rather than in recall. The decision counts of the reported run locate each step in the error space: P1 raises TP from 186 to 199 and reduces FN from 29 to 16 while FP stays at 75, so recall rises from 0.8651 to 0.9256 without

adding a false positive, and P2 and P3 then reduce FP to 68 and 61 with TP unchanged at 199. These counts describe one run per configuration and serve as a case analysis of where each component acts, not as evidence that the same counts recur in every run.

The six-run means are the primary basis for comparing the four configurations. The gap between P0 and P1 (0.0307) is larger than the six-run standard deviation of either configuration, whereas the gap between P2 and P3 (0.0035) is smaller than that spread. The intervals in Table 16 describe run-to-run variation within a configuration and are not a test between configurations. We performed no paired test between adjacent configurations, so we report the ordering of the means and do not claim that every individual step is statistically significant."

Table V: Nested prompt ablation on 1,217 crash reports and six-run stability of each configuration (revised Table 16). Precision, recall, and F1 come from one run per configuration, namely the median-F1 run for P1 and P2 and the formal V3 and V4 endpoints for P0 and P3; mean F1 and standard deviation are computed over six runs for every configuration.

Prompt	Precision	Recall	F1	Mean F1 (6 runs)	SD	95% run-bootstrap interval
P0 (V3)	0.7126	0.8651	0.7815	0.7788	0.0051	[0.7741, 0.7815]
P1	0.7263	0.9256	0.8139	0.8095	0.0063	[0.8046, 0.8139]
P2	0.7453	0.9256	0.8257	0.8260	0.0100	[0.8178, 0.8342]
P3 (V4)	0.7654	0.9256	0.8379	0.8295	0.0085	[0.8220, 0.8355]

Section 5.6, Evaluation of Final-Prompt Sensitivity and LLM

"The cJSON result illustrates how representative-level decisions translate into report-level metrics. Its 78 reports contain six target and 72 non-target reports. Filtering retains 14 reports, which map to four representatives; all six target reports share one representative. With the evidence package and mapping fixed, this representative changes from non-target under V3 to target under V4, and its prediction is propagated to the six group members. The remaining 72 reports keep their non-target decisions. Consequently, TP changes from 0 to 6 and FN from 6 to 0, while FP remains 0 and TN remains 72, yielding the F1 change from 0.0000 to 1.0000. In the P1 configuration reported in Table 16, the same representative already changes to target after the two-hypothesis comparison is added. Thus, the large project-level change reflects

a prompt-sensitive representative decision shared by all six positives, rather than six independent model decisions."

Table VI: Per-library effect of final prompt design under Qwen3-Coder-Flash (revised Table 17).

Library	V3			V4		
	Prec.	Recall	F1	Prec.	Recall	F1
c-ares	0.5000	1.0000	0.6667	0.5000	1.0000	0.6667
cJSON	0.0000	0.0000	0.0000	1.0000	1.0000	1.0000
dlt-daemon	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
expat	0.0000	0.0000	0.0000	1.0000	0.5385	0.7000
faad2	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
isotp-c	0.5333	1.0000	0.6957	0.5333	1.0000	0.6957
jansson	0.3824	1.0000	0.5532	0.6500	1.0000	0.7879
json-c	0.6500	1.0000	0.7879	0.6500	1.0000	0.7879
libmad	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
libmspack	1.0000	1.0000	1.0000	1.0000	1.0000	1.0000
libpcap	0.6667	0.6316	0.6486	0.6667	0.6316	0.6486
libsndfile	0.8667	1.0000	0.9286	0.8667	1.0000	0.9286
tinyxml2	0.3571	0.7692	0.4878	0.3571	0.7692	0.4878
zlib	0.6842	1.0000	0.8125	0.6842	1.0000	0.8125

Section 5.6, Evaluation of Final-Prompt Sensitivity and LLM

"Run-to-run stability. The four prompt configurations defined in the nested prompt ablation were frozen before evaluation and shared across projects. No project names, expected CVE identifiers, or ground-truth labels were inserted into the templates. Project evidence reaches the model through the evidence package rather than through the template, so no template carries a library-specific instruction. We repeated each configuration six times at temperature 0 and report decision variability at both the run and report levels in Table 16. The Precision, Recall, and F1 columns of that table report one run per configuration: the run with the median F1 of the six repetitions for P1 and P2, and the formal V3 and V4 endpoints for P0 and P3. The six-run mean and standard deviation are computed over the same six repetitions for all four configurations, and these means are the values we compare across configurations. The intervals describe run-to-run variation within a configuration; they are not a test between configurations, so we do not use their overlap to judge significance."

Section 5.6, Evaluation of Final-Prompt Sensitivity and LLM

"Table 18 shows that, with the V4 evidence package and differential prompt held constant, the evaluated backends achieve closely matched overall performance, with F1 ranging from 0.8240 to 0.8379 and recall from 0.9163 to 0.9256. The three backends from the DeepSeek, GPT, and GLM families reach 0.8354, 0.8354, and 0.8312, which places them inside this range, so the observed V4 behavior is not specific to the Qwen series. Backend selection can therefore follow deployment requirements while preserving the overall method behavior."

Comment 8: *The title and introduction emphasize connected vehicle scenarios, but the experimental section is conducted only on general third-party open-source libraries, without involving connected-vehicle-specific communication protocols (e.g., CAN, MQTT, DDS), real-time constraints, or security authentication mechanisms. The authors are advised to supplement: (1) specific mapping between the selected libraries and connected vehicle functions, along with security impact analysis; (2) whether connected-vehicle-specific crash patterns (e.g., timing-related crashes, resource contention crashes) were considered; (3) discussion of the method's applicability in the context of automotive-grade certification (e.g., ISO 26262).*

Response: Thank you for this helpful comment. This study takes the security testing of third-party components in connected-vehicle software as its application setting, and we address the three points as follows.

The selected libraries are general-purpose components that can be used in connected-vehicle software, covering diagnostic logging, communication, configuration parsing, and data processing; their use is not limited to connected vehicles. The term "Connected Vehicles" in the title reflects this application setting, which we now state more explicitly in the abstract and the introduction, and we have added the correspondence between the target libraries and the related functions together with the potential security impact.

First, the subject table maps the evaluated libraries to representative connected-vehicle functions. These include diagnostic logging through dlt-daemon, inter-module transport through isotp-c, and networking, configuration and telemetry processing, compression, media processing, and archive handling through the remaining libraries. The accompanying text explains that defects in these components may affect diagnostic records, communication, configuration handling, update processing, and media services.

The above revisions have been incorporated into Section 5.1 (Experimental Setup) and Table 6. The corresponding excerpts are reproduced below.

Table VII: Third-party open-source libraries used in the evaluation (revised Table 6).

Library	Function	APIs	Crash Reports	Deployment Role
c-ares	Asynchronous DNS resolution	27	103	Gateway DNS
cJSON	JSON parsing and generation	60	78	Config/telemetry parsing
dlt-daemon	Diagnostic logging	13	92	In-vehicle diagnostic logging and fault-record storage
expat	XML parsing	12	36	Vehicle configuration and metadata parsing
faad2	AAC audio decoding	12	151	In-vehicle AAC and media decoding
isotp-c	ISO-TP multi-frame transport	13	104	ISO-TP transport between ECUs and vehicle modules
jansson	JSON parsing and generation	12	128	ECU configuration and telemetry serialization
json-c	JSON parsing and generation	12	40	Configuration and telemetry processing in vehicle software
libmad	MPEG audio decoding	12	112	In-vehicle MPEG audio decoding
libmspack	Archive extraction	12	54	Vehicle software package and archive processing
libpcap	Packet capture and filtering	30	115	In-vehicle network monitoring and packet capture
libsndfile	Audio file processing	12	76	Audio and log-signal processing
tinyxml2	XML parsing	13	63	Vehicle configuration and metadata parsing
zlib	Compression and decompression	29	65	OTA-update and diagnostic-log compression
Combined	–	269	1,217	–

Second, the added transport and logging targets, isotp-c and dlt-daemon, extend the evaluation to component-level state management, call order, and resource lifecycle. This experiment focuses on crash adjudication for individual components: we did not construct timing-related or concurrent resource-contention conditions, so these two crash types are not evaluated. The revised text distinguishes this coverage from vehicle-level timing and concurrent resource-contention validation, which require a system-level execution setting.

Third, CrashSage prepares attribution evidence for further engineering inspection. In vehicle-grade software verification or certification processes, it can assist in screening crash reports, reducing repeated analysis, and providing traceable attribution evidence for developers to verify and repair. We distinguish this supporting role from ISO 26262 compliance assessment and automotive certification. A brief clarification in the experimental setup defines this scope, while the abstract and introduction retain connected vehicles as the application scenario.

> Section 5.1, Experimental Setup
> "Stateful API sequences in isotp-c and dlt-daemon extend the component-level evaluation to transport and logging lifecycles. This experiment adjudicates crashes of individual components and does not construct timing-related or concurrent resource-contention conditions, so these two crash types are not evaluated here. Vehicle-level timing failures and concurrent resource contention require separate system-level validation. CrashSage provides crash-attribution evidence for further engineering analysis; ISO 26262 compliance assessment and certification remain distinct from the component-level evaluation reported here."

> Section 5.1, Experimental Setup
> "Faults in these components may affect diagnostic records, ECU-to-module communication, configuration or telemetry processing, update and log handling, and in-vehicle media services."

Comment 9: *Representative selection compresses 258 reports into 57 representative samples, reducing LLM submissions by 77.91%. However, different crash reports may correspond to different vulnerability instances under the same CVE, or different manifestations of the same vulnerability under different inputs. The authors are advised to supplement: (1) analysis of the correspondence between signature matching and CVE uniqueness; (2) whether there is a risk of misclassifying different vulnerabilities as duplicates; (3) discussion of vulnerability diversity lost after deduplication.*

Response: Thank you for this comment. We have added structural and CVE audits to explain the relationship between report counts, representative groups, and vulnerability identities. We address the three questions in turn.

On the correspondence between signature matching and CVE uniqueness, the audit uses three units: representative groups, crash sites, and CVE identifiers. A report belongs to exactly one representative group, and a group belongs to one crash site. We do not assume a fixed hierarchy between crash sites and CVE identifiers, because one CVE may cover several crash sites and reports at one crash site may reference different CVE identifiers. The group and crash-site levels reduce the report count for two different reasons. Many reports collapse into one group when they share a signature, which is the common case because different drivers often reproduce the same crash in the same way. One crash site then spreads over several groups when its reports reach the same function through different call paths, because the signature carries the call stack and a different path produces a different signature. The 215 target reports form 42 groups over 16 crash sites. Of these 215 reports, 80 carry an explicit public CVE identifier, and these 80 reports form 19 groups over 7 CVEs, so one CVE corresponds to several groups. One CVE in tinyxml2 illustrates the relation. Its 13 reports occupy six groups, and all six groups reach the same crash function, so the groups differ in call path rather than in crash site. Signature matching therefore does not reproduce CVE boundaries. It reproduces crash-site boundaries at a finer scale.

On the possibility of merging different vulnerabilities, the audit inspects every group and lists the CVE identifiers and structural crash-site fingerprints of its members. Among the audited groups, none mixes two CVE identifiers, and none of the 42 target groups mixes two structural fingerprints. Pairwise precision is 1.0000 against both references. These observations show consistency with the audited CVE and structural classes; they do not establish that two reports share an identical root cause, and the structural fingerprint serves as an auditable crash-site proxy.

On vulnerability diversity after deduplication, representative selection reduces the 534 retained reports to 96 representatives, a reduction of 82.02%. All 16 crash sites and all 7 CVEs remain represented after selection. The 135 target reports outside the CVE subset are drawn from publicly documented driver-and-PoC cases without an assigned CVE identifier, from the two automotive components added in this revision, and from the completed audit of the previously pending reports. The representative set therefore covers disclosed vulnerabilities, documented cases without an identifier, and newly audited targets.

The above revisions have been incorporated into Section 5.4 (Evaluation of Representative Selection). The corresponding excerpts are reproduced below.

Section 5.4, Evaluation of Representative Selection

"The audits use three units: representative groups, crash sites, and CVE identifiers. A report belongs to exactly one representative group, and a group belongs to one crash site. We do not assume a fixed hierarchy between crash sites and CVE identifiers, because one CVE may cover several crash sites and reports at one crash site may reference different CVE identifiers. The group and crash-site levels reduce the report count for different reasons. Reports that share a signature collapse into one group, which is the common case because different drivers often reproduce the same crash in the same way. A single crash site can spread over several groups when its reports reach the same function through different call paths, because the signature carries the call stack and a different path yields a different signature. One CVE in tinyxml2 illustrates the relation: its 13 reports occupy six groups, and all six groups reach the same crash function, so the groups differ in call path rather than in crash site. Signature matching therefore does not reproduce CVE boundaries; it reproduces crash-site boundaries at a finer scale."

Section 5.4, Evaluation of Representative Selection

"We audit whether representative selection merges distinct target reports in two complementary views. The structural audit covers all 215 target reports and groups them by project, sanitizer bug type, first target-project frame, and crash function. These reports form 42 representative groups over 16 structural fingerprints, and no group mixes two fingerprints, so all 16 structural classes remain represented after selection. The CVE audit covers the 80 reports with explicit public CVE identifiers. These reports form 19 groups over 7 CVEs, which shows that one CVE corresponds to several groups; we found no group whose members reference different CVE identifiers, and all 7 CVEs remain represented after selection. Neither audit merges distinct target reports, and pairwise precision against both references is 1.0000. The structural fingerprint is an auditable crash-site proxy rather than an independent manual root-cause label, so these results show consistency with the audited classes and do not establish that two reports share an identical root cause."

Section 5.4, Evaluation of Representative Selection

"The 80 CVE-linked reports are a subset of the 215 target reports; the remaining 135 reports comprise 66 author-confirmed public driver+PoC cases without a CVE identifier, 43 targets from dlt-daemon and isotp-c, and 26 targets from the completed audit."

Cybersecurity
Manuscript ID: CYSE-D-26-01344
Authors' Response to Reviewer 2

Comment 1: *Clarify the connection between the connected vehicles and the technical contribution. Section 5.1 explicitly focuses on third-party libraries rather than a complete vehicle software stack, and Table 6 lists their representative deployment roles. However, the technical challenges, method design, and evaluation mainly reflect general crash analysis of open-source libraries. It remains unclear what automotive-specific requirements motivate or affect the design of CrashSage. If the connected vehicles is central to the paper, representative software stacks or realistic integration configurations would strengthen the evaluation. Otherwise, the paper could position CrashSage as a general crash-adjudication approach and present automotive software as one application scenario. The corresponding scope and applicability limitations should also be clarified in Section 6.*

Response: Thank you for this helpful comment. We have clarified the distinction between the application setting and the technical contribution. Connected-vehicle software is the application scenario considered in this study; the technical problem addressed by CrashSage is the uncertain correctness of LLM-generated fuzz drivers. The title is retained to identify this application scenario, while the abstract states this distinction explicitly.

The method uses sanitizer evidence, stack-frame roles, representative matching, and reconstructed execution context. These designs address driver/component attribution and do not depend on a vehicle-specific protocol or an on-vehicle execution environment. The evaluated libraries provide the concrete application workload, and the subject table records their representative roles in connected-vehicle software, covering diagnostic logging, communication, configuration parsing, and data processing.

Section 6 now specifies component-level crash adjudication as the evaluation scope and distinguishes it from full-stack, ECU-integration, and on-vehicle validation. We appreciate the suggestion and have aligned the abstract and the evaluation scope with this application setting.

The above revisions have been incorporated into the Abstract and Section 6 (Limitations). The corresponding excerpts are reproduced below.

Abstract

"Connected-vehicle software relies on third-party open-source libraries, making it a relevant application setting for studying crash adjudication in LLM-assisted fuzzing."

Section 6, Limitations

"The evaluation covers third-party-library components of connected-vehicle software. Full vehicle stacks, ECU integration, and on-vehicle deployment are not exercised, so the results describe component-level crash adjudication rather than vehicle-level behavior. The method uses sanitizer evidence, stack-frame roles, representative matching, and reconstructed execution context; these designs address driver/component attribution and do not depend on a vehicle-specific protocol or an on-vehicle execution environment."

Comment 2: *Expand the diversity of experimental targets. The current evaluation covers 12 libraries, mainly involving structured-data parsing, media decoding, networking, and compression/archive processing. Although these targets are practically relevant, broader API usage patterns would provide stronger evidence of generalization. In particular, the authors could consider targets involving state management, resource lifecycles, invocation-order constraints, or other complex API contracts. The paper should also clarify the target-selection criteria and deployment limitations.*

Response: Thank you for this valuable comment. The comment raises four points: the target-selection criteria, the API usage patterns that the current targets exercise, the addition of targets with state and lifecycle contracts, and the deployment limitations. We address them in turn.

On the selection criteria, a library enters the evaluation if it is publicly available, is relevant to the connected-vehicle supply chain, builds under ASan+UBSan within the CKGFuzzer workflow, and produces crashes that can be reproduced. The final set contains 14 libraries, and all 12 libraries of the previous version are retained in it.

On the API usage patterns, we now quantify the interaction that the generated drivers exercise rather than the functional category of each library. The 14 projects contain 186 generated drivers, between 7 and 22 per project, and the 1,217 reports are distributed over them at 6.54 reports per driver on average. The median number of distinct target APIs per driver ranges from 2 to 8 across projects, and 179 of the 186 drivers (96.24%) call at least two distinct target APIs. 168 drivers (90.32%) include at least one initialization or cleanup call. The presence of such a call does not show that a driver covers both ends of

a lifecycle, so we no longer describe the drivers in general as following a setup-to-release sequence; the ordered call sequences recorded for dlt-daemon and isotp-c are the concrete lifecycle examples. We have also corrected the three derived values in this paragraph: 179 of 186 is 96.24%, 168 of 186 is 90.32%, and 1,217 reports over 186 drivers is 6.54 reports per driver.

On the state and lifecycle targets, we added `isotp-c` and `dlt-daemon` to cover transport and logging components. The recorded drivers exercise ordered calls and resource lifecycles. For example, the `dlt-daemon` driver calls `dlt_filter_init`, `dlt_filter_load`, `dlt_filter_add`, `dlt_filter_find`, `dlt_filter_delete`, and `dlt_filter_free` in order, and the `isotp-c` driver calls `isotp_receive`, `isotp_continue_receive`, `allocate`, and `free_allocated`. These examples document component-level interaction patterns; vehicle-level timing and concurrency remain separate validation settings.

On the deployment scope, Section 6 identifies third-party-library components as the evaluation layer and distinguishes this setting from full vehicle-stack, ECU-integration, and on-vehicle validation.

The above revisions have been incorporated into Section 5.1 (Experimental Setup). The corresponding excerpts are reproduced below.

> Section 5.1, Experimental Setup
> "All 14 libraries are publicly available, are relevant to the connected-vehicle supply chain, build under ASan+UBSan within the CKGFuzzer workflow, and produce crashes that can be reproduced; the final set contains 14 libraries and retains all 12 libraries of the previous version. We characterize exercised API diversity by recording multi-API calls and ordered lifecycle calls involving initialization, creation, opening, allocation, receiving, use, and cleanup. The 14 projects contain 186 generated drivers, between 7 and 22 per project, and the 1,217 reports are distributed over them at 6.54 reports per driver on average. The median number of distinct target APIs per driver ranges from 2 to 8 across projects, 179 of the 186 drivers (96.24%) call at least two distinct target APIs, and 168 drivers (90.32%) include at least one initialization or cleanup call. The presence of such a call does not show that a driver covers both ends of a lifecycle, so we do not attribute a complete setup-to-release pattern to the drivers in general. The drivers recorded for dlt-daemon and isotp-c show ordered lifecycle use directly: the dlt-daemon driver calls `dlt_filter_init`, `dlt_filter_load`, `dlt_filter_add`, `dlt_filter_find`, `dlt_filter_delete`, and `dlt_filter_free` in order, and the isotp-c driver calls `isotp_receive`, `isotp_continue_receive`, `allocate`, and `free_allocated`.

Faults in these components may affect diagnostic records, ECU-to-module communication, configuration or telemetry processing, update and log handling, and in-vehicle media services."

Comment 3: *Clarify the effect of rule-based filtering. Table 9 shows that all 146 ground-truth positive reports are retained after filtering, yet Tables 10 and 11 show noticeable Recall differences between V0 and V1, particularly for expat, json-c, and libpcap. The paper explains that the false negatives in V1 come from subsequent LLM decisions rather than filtering, but it remains unclear why retained positive reports receive different predictions between the two configurations. Please clarify whether normalization, triage information, or other model inputs differ between V0 and V1. If the effective inputs and model settings are otherwise unchanged, repeated runs would help determine whether model variability contributes to these differences.*

Response: Thank you for this careful reading. The comment raises three questions: which model inputs differ between V0 and V1, why retained target reports receive different predictions, and whether run-to-run variation contributes to these differences.

V0 uses the original crash report. V1 additionally uses the normalized crash report and the evidence produced by the rule-guided stage. Both configurations use the same model, the same temperature, and the same output schema. Rule-guided filtering removes 683 reports, all of which are non-target, and retains all 215 target reports. V1 improves precision from 0.2817 to 0.5894, recall from 0.7023 to 0.7209, and F1 from 0.4021 to 0.6485. The filtering step of V1 therefore retains every target report, but the LLM input of V1 differs from that of V0: in addition to the original report, the normalized report and the rule evidence are supplied. Retained reports can therefore still receive different predictions, and the change in recall is not caused by target reports being filtered out.

To isolate the effect of normalization, we introduced V0|N, which retains V0's original input and adds only the normalized crash report. V0|N increases recall from 0.7023 to 0.7814 and identifies 17 additional target reports, while precision remains nearly unchanged. The normalized report presents crash type, access behavior, stack order, and project location in a consistent form. This makes the relevant evidence easier to locate and improves target recognition.

The difference between V0|N and V1 arises from two stages. First, filtering removes a large number of non-target reports before the final judgment. This reduces false target decisions and accounts for much of the increase in precision and F1. Second, V1 gives greater weight to driver-side conditions in some retained cases. When the driver contains

an explicit check or supplies an invalid value, V1 more often places responsibility on the driver. This reduces target recall relative to V0|N.

Relative to V0|N, V1 therefore reduces false positives but also increases some missed detections. We select V1 as the basis of the subsequent pipeline because it reduces both the number of reports to be analyzed and the number of false positives while achieving better overall attribution, which matches the goal of reducing the burden of inspecting invalid reports. Its F1 is 0.6485, compared with 0.4164 for V0|N, and relative to the original V0 both precision and recall improve.

The per-library results are consistent with this explanation. In cJSON, expat, and zlib, the affected cases contain driver-side checks or invalid driver values. V1 gives greater weight to these conditions, so some reports treated as target cases by V0 are instead treated as driver cases. These changes reduce F1 for the corresponding projects. In isotp-c, the invalid bit index comes from the driver, but the crash occurs in target code. V1 gives greater weight to the target code, which raises recall but also increases false positives and lowers F1. These project-level outcomes therefore reflect a shift in where responsibility is assigned, while the full-population precision and F1 remain higher for V1.

We also repeated V1 with the same model, the same temperature, and the same frozen inputs. Table VIII compares the formal run with the repeated run. The repeated run changes aggregate F1 by 0.0012 and supports the same conclusion; this variation is considerably smaller than the V0–V1 difference. The repeated-run comparison is reported in this letter only, and the revised manuscript reports the formal V1 result.

The above revisions have been incorporated into Section 5.3 (Evaluation of Rule-Guided Crash Attribution). The corresponding excerpts are reproduced below.

Table VIII: Formal and repeated runs of V1 on the same frozen inputs.

Run	TP	FP	TN	FN	Precision	Recall	F1
Formal V1	155	108	894	60	0.5894	0.7209	0.6485
Repeated V1	156	111	891	59	0.5843	0.7256	0.6473

We have revised the manuscript to separate the effects of normalization, filtering, and rule-derived evidence. The revised discussion no longer attributes the complete V0–V1 improvement solely to the added evidence. It reports that normalization improves target coverage, filtering provides the principal precision gain, and the rule-derived evidence changes the final driver/target judgment.

To clarify it, we have improved the paper as follows.

Section 5.3, Evaluation of Rule-Guided Crash Attribution

"To isolate the contribution of crash-report normalization, we further evaluate V0|N, which retains V0's original input and adds only the normalized crash representation. V0|N increases recall from 0.7023 to 0.7814 and identifies 17 additional target reports, while precision remains essentially unchanged. In V0, crash type, access behavior, stack order, and project location are embedded in a long and redundant sanitizer report. The model must first distinguish relevant evidence from addresses and repeated runtime messages. V0|N presents these fields in a consistent structure and preserves their order. This reduces ambiguity in the execution path, allowing the model to connect the driver call to the target operation more reliably and recover target cases that V0 misses. The comparison between V0|N and V1 can be attributed to two distinct stages. Relative to V0|N, V1 achieves higher precision and F1 but lower recall. The precision difference is primarily a filtering effect, because rule-guided crash attribution removes 683 non-target reports before the final judgment and thereby eliminates a substantial number of false-positive opportunities. The recall difference arises at the subsequent judgment stage. Since rule-guided crash attribution removes only non-target reports, the reduction in recall is associated with decisions on the retained target cases. When the driver contains an explicit assertion or provides an invalid value, V1 assigns greater weight to driver responsibility. This explains the reduced recall in cJSON, expat, and zlib. In target-side memory operations reached through invalid driver inputs, V1 assigns greater weight to the target crash site, which increases target detections but also introduces additional false positives, as observed in isotp-c. These effects jointly explain the overall trade-off. The structured report improves target coverage, filtering provides the principal gain in precision, and the rule-derived evidence changes the final assignment of responsibility between the driver and the target."

Comment 4: *Strengthen the evaluation of model stability and generalization. The experiments use temperature 0, but deterministic decoding does not necessarily establish run-to-run stability when using a cloud-hosted LLM. Repeating the main experiments and reporting the variation in key metrics would make the conclusions more robust. In addition, Table 17 evaluates three Qwen models under the same evidence package and prompt, but all belong to the same model family. If feasible, including a representative model from another family would provide stronger evidence that the observed benefits are not specific to Qwen models.*

Response: Thank you for this valuable comment. The comment asks for two additions:

repeated runs that quantify run-to-run variation, and a backend from another model family. We address them in turn.

On run-to-run stability, we repeat the main experiment six times at temperature 0 and report the variation in the key metric. For the full CrashSage configuration, the mean F1 is 0.8295 with a standard deviation of 0.0085, and the 95% run-bootstrap interval is [0.8220, 0.8355]. Across the four repeated configurations, the standard deviation lies between 0.0051 and 0.0100, and the number of reports whose decision changes across the six runs is between 9 and 23 out of 1,217. The conclusions therefore hold under repetition of the same configuration.

On other model families, we repeat the complete V4 experiment with five additional backends, three of which belong to other model families: `deepseek-flash`, `gpt-5.6-sol`, and `glm-5.3-flash`. The representatives, reconstructed contexts, prompt, representative mapping, and ground-truth labels are held fixed, and only the final LLM is replaced. F1 ranges from 0.8240 to 0.8379 and Recall from 0.9163 to 0.9256 across the six backends. The three non-Qwen backends reach 0.8354, 0.8354, and 0.8312, which places them inside this range.

The above revisions have been incorporated into Section 5.6 (Evaluation of Final-Prompt Sensitivity and LLM), Table 16, and Table 18. The corresponding excerpts are reproduced below.

Section 5.6, Evaluation of Final-Prompt Sensitivity and LLM

"Run-to-run stability. The four prompt configurations defined in the nested prompt ablation were frozen before evaluation and shared across projects. No project names, expected CVE identifiers, or ground-truth labels were inserted into the templates. Project evidence reaches the model through the evidence package rather than through the template, so no template carries a library-specific instruction. We repeated each configuration six times at temperature 0 and report decision variability at both the run and report levels in Table 16. The Precision, Recall, and F1 columns of that table report one run per configuration: the run with the median F1 of the six repetitions for P1 and P2, and the formal V3 and V4 endpoints for P0 and P3. The six-run mean and standard deviation are computed over the same six repetitions for all four configurations, and these means are the values we compare across configurations. The intervals describe run-to-run variation within a configuration; they are not a test between configurations, so we do not use their overlap to judge significance."

Table IX: Nested prompt ablation on 1,217 crash reports and six-run stability of each configuration (revised Table 16). Precision, recall, and F1 come from one run per configuration, namely the median-F1 run for P1 and P2 and the formal V3 and V4 endpoints for P0 and P3; mean F1 and standard deviation are computed over six runs for every configuration.

Prompt	Precision	Recall	F1	Mean F1 (6 runs)	SD	95% run-bootstrap interval
P0 (V3)	0.7126	0.8651	0.7815	0.7788	0.0051	[0.7741, 0.7815]
P1	0.7263	0.9256	0.8139	0.8095	0.0063	[0.8046, 0.8139]
P2	0.7453	0.9256	0.8257	0.8260	0.0100	[0.8178, 0.8342]
P3 (V4)	0.7654	0.9256	0.8379	0.8295	0.0085	[0.8220, 0.8355]

Table X: Validation results of different LLM backends under the same V4 configuration (revised Table 18).

LLM Backend	Precision	Recall	F1
Qwen3-Coder-Flash (default V4)	0.7654	0.9256	0.8379
Qwen-Flash	0.7425	0.9256	0.8240
Qwen3.8-Flash	0.7636	0.9163	0.8330
DeepSeek-Flash	0.7645	0.9209	0.8354
GPT-5.6-Sol	0.7645	0.9209	0.8354
GLM-5.3-Flash	0.7606	0.9163	0.8312

Comment 5: *Clarify the CKGFuzzer baseline and comparison with prior work. Section 5.1 states that crash reports are collected following the fuzz-driver generation and fuzzing workflow of CKGFuzzer, while Table 7 labels V0 as the "CKGFuzzer baseline" and describes it as having no filtering, selection, or context reconstruction and using a clean direct prompt. It is therefore unclear whether V0 represents the complete crash-analysis procedure of CKGFuzzer or only uses its upstream workflow to generate crash reports. The authors should clearly specify which CKGFuzzer components are retained or modified. If V0 does not include its original crash-analysis procedure, a separate comparison with the complete method would provide a fairer baseline. Comparisons with other representative methods directly related to crash attribution or false-positive identification would also help establish the advantages of CrashSage beyond its internal ablations.*

Response: Thank you for this valuable comment. The comment raises four points: what V0 contains, which CKGFuzzer components are retained or added, whether a separate comparison with the complete method is needed, and how CrashSage compares with other representative methods. We address them in turn.

On what V0 contains, V0 runs the CKGFuzzer crash-cause analysis stage on the same inputs that CKGFuzzer uses, not a reuse of the upstream workflow with an empty analysis stage. The inputs of that stage, namely the crash report, the generated driver and the APIs it calls, the error patterns extracted from their source, and the retrieved CWE entries, are unchanged. The upstream driver generation and fuzzing workflow is shared by all variants, and the crash-analysis stage of CKGFuzzer is active in V0.

On the retained and added components, the four components evaluated in V1–V4, namely rule-guided filtering, representative selection, context reconstruction, and the differential prompt, are CrashSage additions and are disabled in V0. The text accompanying the configuration table now states that V0 runs the CKGFuzzer crash-analysis stage with its original inputs and adds none of the four CrashSage stages, so the text identifies which components are retained and which are added. Because the upstream workflow is held fixed across V0–V4, the comparison measures the added stages on top of that stage.

On the comparison with the complete method, V0 already contains the CKGFuzzer crash-analysis stage with its original inputs. The V0–V4 endpoints therefore provide this comparison directly, and we have made that relationship explicit in the accompanying text.

On other representative methods, we add two external references. A raw-report LLM baseline passes each of the 1,217 reports directly to the same model with a minimal direct instruction, without filtering, representative selection, reconstructed context, or the differential prompt. It performs the same binary decision as CrashSage, so it is the direct reference for false-positive identification. It obtains Precision 0.2010, Recall 0.5488, and F1 0.2943, against 0.7654, 0.9256, and 0.8379 for CrashSage. We also apply ReBucket to the same 534 reports retained after rule-guided crash attribution, as the external reference for the selection stage. ReBucket selects 103 cluster representatives and CrashSage selects 96, corresponding to reductions of 80.71% and 82.02%. At the pairwise level, ReBucket preserves 1,450 of the 1,476 report pairs that CrashSage groups together, and the pairwise Jaccard agreement is 0.9824.

The above revisions have been incorporated into Section 5.1 (Experimental Setup), Section 5.4 (Evaluation of Representative Selection), and Table 9. The corresponding excerpts are reproduced below.

Section 5.1, Experimental Setup

"All variants share the same base configuration, which is the crash-analysis stage of CKGFuzzer: the crash report, the generated driver and the APIs it calls, the error patterns extracted from their source, and the retrieved CWE

entries. V0 therefore runs the crash-analysis stage of CKGFuzzer with its original inputs unchanged and adds none of the four CrashSage stages. V1–V4 keep this configuration unchanged and add the stages listed in Table 7. The V0–V4 comparison therefore measures the incremental effect of these added stages while holding the fuzz-driver generation and fuzzing workflow of CKGFuzzer fixed and evaluating all variants on the same crash reports."

Table XI: End-to-end comparison between raw-only LLM analysis and complete CrashSage V4 (revised Table 9).

Method	LLM submissions	Precision	Recall	F1
Raw-only LLM	1,217	0.2010	0.5488	0.2943
CrashSage V4	96	0.7654	0.9256	0.8379

Section 5.4, Evaluation of Representative Selection

"To compare representative selection directly, we apply ReBucket to the same 534 reports retained after rule-guided crash attribution. CrashSage selects 96 representatives, whereas ReBucket selects 103 cluster representatives, corresponding to reductions of 82.02% and 80.71%, respectively. Thus, ReBucket requires seven additional LLM submissions relative to CrashSage on this workload. At the pairwise level, ReBucket preserves 1,450 of the 1,476 report pairs that CrashSage groups together and splits the remaining 26, whereas no pair is grouped by ReBucket alone. The pairwise Jaccard agreement is 0.9824."

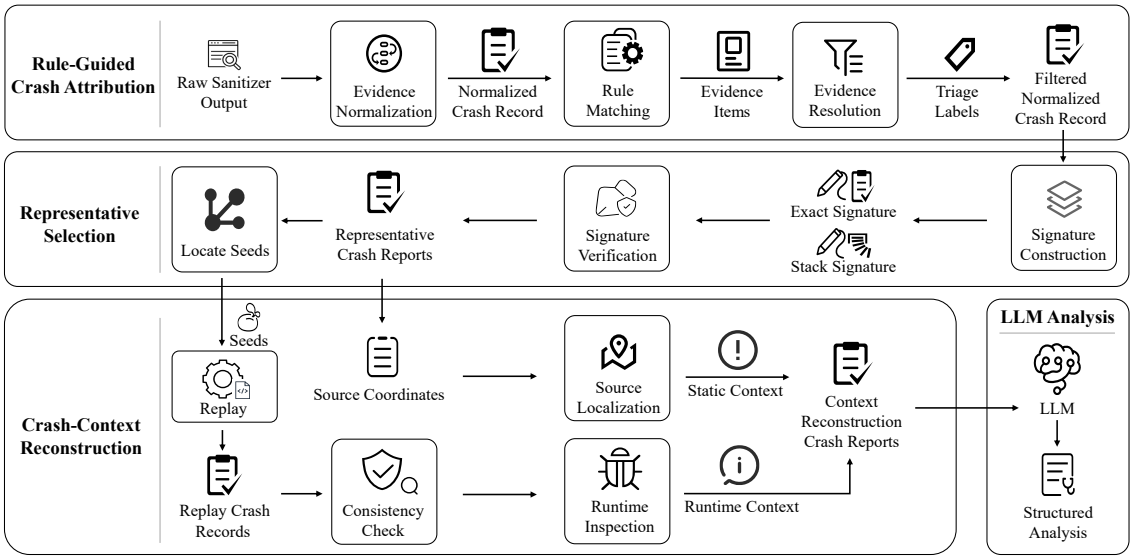
Comment 6: *Improve figure readability. Some figures, particularly the pipeline diagrams, contain text that is difficult to read at normal viewing scale. Increasing the font size and simplifying the layout would improve readability. Vector graphics should be used where appropriate.*

Response: Thank you for this valuable comment. We redrew the pipeline figure and enlarged its labels.

The figure is now inserted at the full text width instead of 0.60 linewidth, which removes most of the reduction in label size at print, and the panels are widened so that the labels no longer overlap. The four stages and their order are unchanged. The revision brings the raw sanitizer output into the first stage so that the panel boundaries match the four processing stages, keeps the connecting arrows, and moves the explanatory text into the caption, so the pipeline shows the same four stages in a simpler layout.

On vector graphics, Fig. 2 contains no raster element and every label is vector text, so it stays sharp at any zoom level and at print resolution. Fig. 1 keeps its icon set with vector text labels, and the result figures are regenerated as vector PDFs. The final insertion width of each figure is checked in the typeset paper.

The above revisions have been incorporated into Figure 2. The corresponding excerpts are reproduced below.



Revised Figure 2.